

ISEN

ALL IS DIGITAL!

OUEST

Année 2024-2025

Electronique numérique

Travaux pratiques Linux Embarqué

Préalable

Matériel nécessaire

- Boîte de rangement contenant :
 - Carte Beaglebone (rev. $\geq$ A6) <http://beagleboard.org/Products/BeagleBone+Black/> et équipée :
 - d'une carte mikroBUS cape
 - d'un module ISEN sur le slot 1
 - Alimentation 5V pour la carte Beaglebone
- Carte MicroSD de 16 GB avec adaptateur Carte SD numéroté
- 1 cordon RJ45 pour la carte Beaglebone

Tout le matériel doit être rendu et rangé en fin de chaque séance de TP.

- PC portable étudiant connecté au réseau ISEN

IMPORTANT : veuillez noter et conserver ces numéros et mots de passe pour les prochains TPs :

- Le numéro de la carte BeagleBone afin de conserver la même adresse IP (chaque carte ayant une adresse IP différente)
- Le numéro de votre poste de travail afin de conserver la même place en TP
- L'adresse IP, les identifiants et mots de passe de votre machine virtuelle
- Le numéro de la Carte MicroSD de 16 GB avec adaptateur Carte SD.

Objectifs

Les objectifs de cette série de TP sont les suivants :

- Configurer les outils de développement et la carte BeagleBone
- Aborder le développement de programmes en langage C pour BeagleBone
- Utiliser la compilation croisée pour développer sur PC Windows, Linux ou macOS des programmes fonctionnant sur BeagleBone/Linux
- Renforcer ses connaissances de Linux en utilisant des commandes shell dans un terminal

Vous réaliserez tout d'abord des programmes simples mettant en œuvre différents périphériques de la carte (ports E/S (GPIO), ADC et PWM). Cela vous familiarisera avec la BeagleBone Black et l'OS Linux embarqué. Ensuite, vous réaliserez des programmes plus complexes en vous appuyant sur les différents programmes que vous aurez déjà développés.

1 TP1 - Configuration et prise en main d'Eclipse

1.1 Outils de développement Eclipse/C/C++ pour microcontrôleurs ARM Cortex A8

Une machine virtuelle sous Linux Debian 10 avec une interface graphique LXDE a été créée sur un serveur de l'ISEN pour chaque étudiant. Vous conserverez cette machine virtuelle pour l'ensemble des TP.

Cette option a été privilégiée car elle ne nécessite pas d'installer les outils de développement sur votre PC et les enseignants peuvent facilement accéder à distance à votre environnement pour vous aider.

1.2 Connexion des cartes BeagleBone

Ces différentes opérations sont à effectuer dans l'ordre :

- Insérer la carte MicroSD (sérigraphie vers le haut) dans la carte BeagleBone (alimentation coupée). Il s'agit d'un connecteur MicroSD de type "push-push", il faut appuyer pour insérer la carte et appuyer de nouveau pour l'enlever, on entend un clic lorsque la carte est bien insérée. Faire attention à ne pas mettre la carte MicroSD à l'envers et à ne pas forcer à l'insertion.
- Suivant les salles de TP la procédure de connexion des cartes BeagleBone au réseau est différente :
 - Sur le site de Brest :
 - En S230 : connecter le cordon RJ45 entre la carte BeagleBone et le switch Ethernet.
 - En S334 : connecter le cordon RJ45 « bleu » entre la carte BeagleBone et le port dont le numéro est impair (pastille bleue).
 - En S335 : connecter le cordon RJ45 « bleu » entre la carte BeagleBone et le switch réseau BeagleBone.
 - Sur le site de Caen :
 - En salle 37 du Lycée Ste-Marie : connecter le cordon RJ45 entre la carte BeagleBone et le port STM32 sur votre table.
 - Sur le site de Nantes :
 - En A1-48 : connecter le cordon RJ45 entre la carte BeagleBone et le switch Ethernet.
- **Il faut impérativement utiliser la connexion WiFi entre votre portable et le réseau ISEN.**
- Connecter l'alimentation à la carte BeagleBone et au secteur, le démarrage de la carte nécessite 2 minutes environ, des petites LED de couleur bleue doivent clignoter dès la mise sous-tension.

En fin de séance, avant de tout déconnecter, vous devez arrêter la carte BeagleBone en tapant la commande "halt" dans une fenêtre "terminal".

1.3 Adresses IP des cartes BeagleBone (VM)

Les adresses IP des cartes BeagleBone sont au format suivant :

- 10.10.49.1xx pour Brest,
- 10.30.49.1xx pour Nantes,
- 10.40.49.1xx pour Caen

où xx représente le numéro de la boîte de rangement. Exemple : la carte BeagleBone BB-SE24 correspond à l'adresse IP 10.10.49.124 si cette carte est branchée sur le réseau de Brest. Les cartes sont accessibles en SSH.

1.4 Connexion à votre machine virtuelle (VM)

Via un navigateur Internet compatible HTML5 (ex : Firefox, Edge, Safari, Chrome. . .) accéder à votre VM en saisissant l'adresse au format suivant :

- `http://10.10.53.100+yy:8080` pour Brest,
- `http://10.10.53.150+yy:8080` pour Nantes,
- `http://10.10.53.180+yy:8080` pour Caen.

où yy représente le numéro du poste de travail où vous vous êtes installés lors de la 1ère séance. Pour les 3 séances de TP, vous travaillerez toujours sur le même poste de travail et sur la même VM afin de retrouver votre travail. Exemple : pour le poste de travail "Poste 12" sur Brest l'adresse de la VM correspondante sera `http://10.10.53.112:8080`.

Le login et mot de passe vous seront donnés par les encadrants.

Le "copier/coller" entre la VM et votre PC se fait via le menu accessible par la combinaison des touches "CTRL-ALT-SHIFT".

Pour une meilleure expérience utilisateur et une meilleure définition vous pouvez passer en plein écran. Suivant la résolution de votre portable vous pouvez adapter la résolution du bureau Linux.

Les VM restent accessibles en dehors des TP depuis le réseau ISEN en WiFi.

1.5 Configuration initiale de la carte BeagleBone qui a été réalisée

Cette partie est uniquement à faire une seule fois en mode binôme.

— Connectez-vous en ssh sur la carte BeagleBone (default username :password is [debian :temppwd]) en ouvrant une fenêtre terminal et en tapant la commande :

`ssh debian@adresse_ip_de_votre_carte_BeagleBone` Le mot de passe est temppwd

Lors de la première connexion ssh vous devez accepter la question posée « are you sure you want to continue connecting (yes/no) ? »

— Vérifier avec la commande `uname -a` que la carte a bien démarré sur le noyau 5.4.x et Debian 10 Buster « Linux BeagleBone 5.4.38-ti-r8 1Buster SMP PREEMPT Sat May 9 09 :53 :15 UTC 2020 armv7l GNU/Linux »

— Passer en root avec la commande `sudo bash` (mot de passe identique)

— Autoriser les connexions ssh avec l'identifiant root. Pour cela vous devez modifier le fichier `/etc/ssh/sshd_config`. Vous pouvez le faire en éditant manuellement le fichier `/etc/ssh/sshd_config` avec l'éditeur texte de votre choix soit vim ou nano, exemple `nano /etc/ssh/sshd_config` puis remplacer la ligne `PermitRootLogin prohibit-password` Par : `PermitRootLogin yes` Ne pas oublier d'enlever le en debut de ligne, qui est un commentaire précisant que le choix par défaut est d'interdire les connexions ssh en root.

— Relancer le daemon `ssh` `systemctl restart sshd` afin de prendre en compte le changement de configuration

— Changer le mot de passe root avec la commande `passwd` (choisir le mot de passe root pour le compte root)

— Fermer la session ssh root avec la commande `exit` ou faire contrôle+d puis se déconnecter de la session ssh debian avec la commande `exit` et se reconnecter en ssh avec le compte root à la carte BeagleBone afin de tester l'accès root depuis ssh

— Redimensionner la partition de la MicroSDCard car l'image Linux fournie est basée sur une

partition de 4 Go alors que la carte MicroSDCard a une capacité de 16 Go. Suivre la diapositive du cours qui indique la procédure. Pour rappel, aller dans `/opt/scripts/tools/` ensuite faire "df-k" constater les tailles utilisées. Exécuter la commande `/opt/scripts/tools/grow_partition.sh` puis redémarrer la carte BeagleBone avec la commande `reboot`. Vérifier avant et après le redémarrage, la taille et l'espace disponible avec la commande `df -k` de la partition `/` (device `/dev/mmcblk0p1`). `df` est l'abréviation de disk free.]

- Installer et configurer la localisation apt install locales
- Vérifier la date avec la commande `date`. Si besoin, configurer le fuseau horaire `timedatectl set-timezone Europe/Paris`.

1.6 Configuration de l'IDE Eclipse

Pour faciliter le développement sur la carte cible BeagleBone nous allons utiliser l'éditeur de texte de l'IDE Eclipse avec le plugin "Remote System Explorer" pour l'accès distant de la carte cible BeagleBone. Cette configuration présente plusieurs avantages :

- Environnement très utilisé dans les milieux académiques et industriels
- Environnement complet avec de nombreuses extensions
- Environnement unique pouvant être utilisé dans de nombreux langages de programmation
- Utilisation pour du développement en natif ou en compilation croisée
- Convivialité et fonctionnalité de l'éditeur : complétion, analyse coloration syntaxique
- Parfaite intégration du développement sur la carte cible : accès au shell, accès au debugger...
- Solution open source largement éprouvée

1.6.1 Test préalable de la connexion avec la carte BeagleBone

Ouvrir une fenêtre terminal depuis la machine virtuelle en cliquant sur l'icone LXTerminal.

Vérifier que votre carte BeagleBone répond bien aux pings avec la commande :

```
ping adresse_ip_de_votre_carte_BeagleBone
```

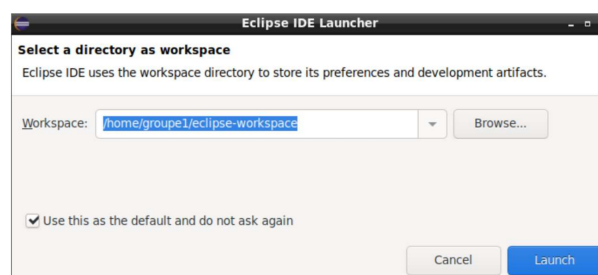
Si ce n'est pas le cas vérifier les branchements puis redémarrer la carte en appuyant sur le bouton reset.

Tester également la connexion ssh en root avec la commande :

```
ssh root@adresse_ip_de_votre_carte_BeagleBone
```

1.6.2 Installation du plugin Remote Sytem Explorer

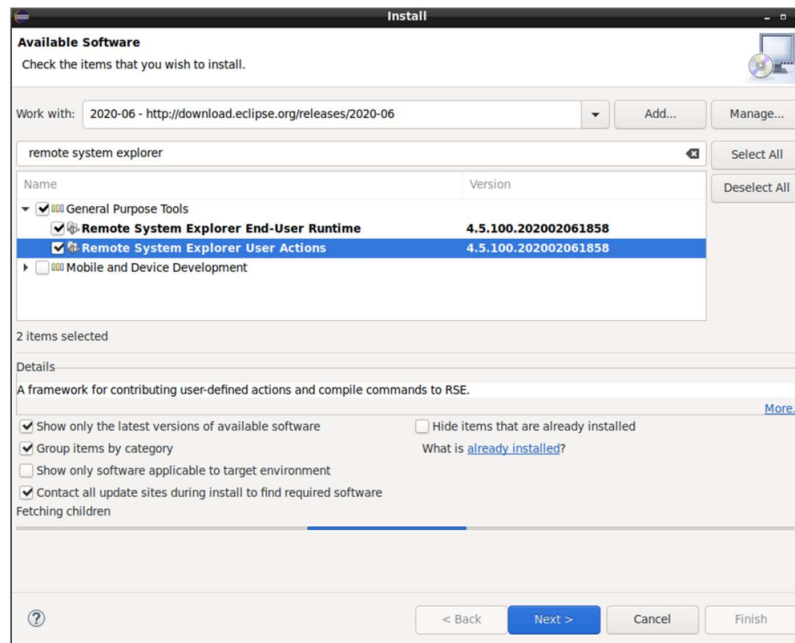
- Démarrer Eclipse.



- Choisir le workspace par défaut.
- Fermer la fenêtre Welcome

- Sous Eclipse, aller dans le menu "Help" puis "Install New Software" puis sélectionner dans le menu déroulant "2020-06 – <http://download.eclipse.org/releases/2020-06>" puis sélectionner les plugins "Remote System Explorer User Actions", "Remote System Explorer End-User Runtime".

Pour faciliter et accélérer cette recherche vous pouvez mettre "Remote System Explorer" comme filtre et les deux paquets se trouveront dans "General Purpose Tools".



- Next
- Next
- Cocher "I accept the terms of the license agreements"
- Finish
- Attendre quelques minutes que l'installation soit terminée
- Restart Now

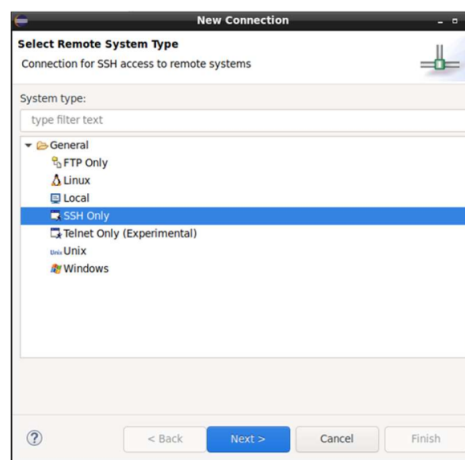
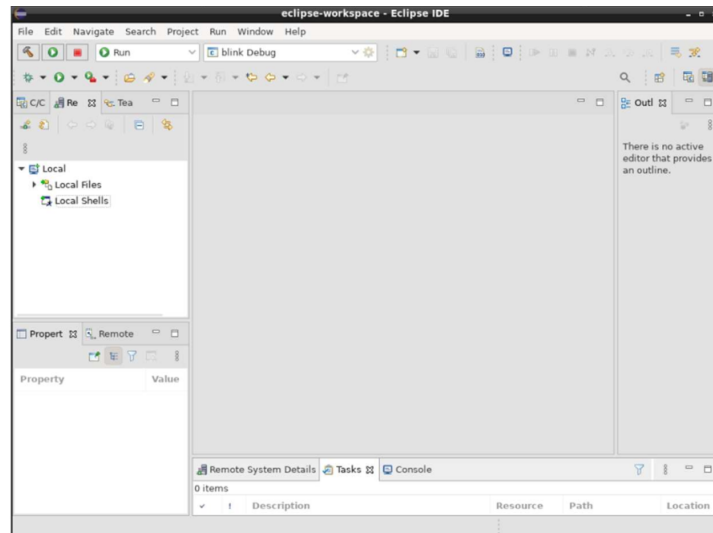
1.6.3 Configuration d'Eclipse et des plugins pour le développement sur la carte cible BeagleBone

- Menu *Window > Perspective > Open Perspective > Other* puis *Remote System Explorer*
- Open

Vous devez obtenir la fenêtre suivante (si besoin cliquer sur les icones à gauche et/ou agrandir la fenêtre) :

1.6.4 Définir la connexion avec votre carte BeagleBone

- Dans l'onglet "Remote Systems" à gauche de l'écran, vous devez créer une connexion en cliquant sur  ("Define a connection to remote system") puis sélectionner *SSH Only*.



— Next

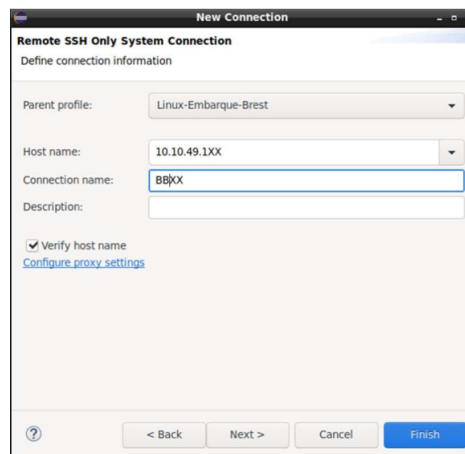
— Ajouter les données pour votre carte cible BeagleBone : Host name (adresse IP), connection name, description (optionnel) puis valider avec le bouton "Finish". (Vous avez chacun un numéro de carte différent)

— Finish

Vous devriez voir apparaître deux éléments : "sftp files", "ssh shells".

"sftp files" donne accès à l'arborescence de votre carte cible BeagleBone, "My home" est le répertoire home du compte Linux que vous avez renseigné. Cela permettra d'éditer les fichiers de votre carte cible BeagleBone depuis Eclipse.

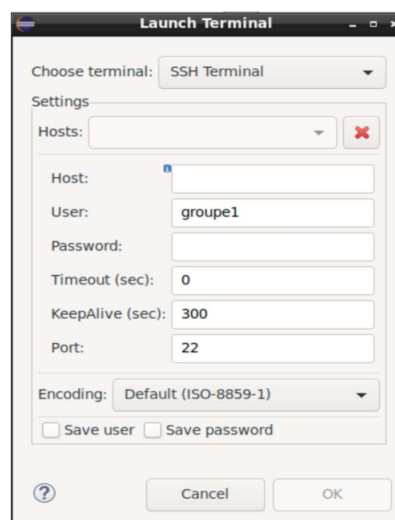
"ssh shells" donne un accès au shell de votre carte cible BeagleBone (menu accessible par clic droit). Cela permettra d'exécuter des commandes Linux, mais le plus pratique est de lancer l'application TM Terminals.



1.6.5 Lancement d'une fenêtre terminal (TM Terminal)

Le lancement d'une fenêtre terminal peut se faire en cliquant sur l'icône , puis choisir *SSH Terminal*. Pour information, la fenêtre Terminal se trouve aussi dans Window puis Show View puis Terminal.

Attention : le champ *User* est renseigné, il faut le modifier.



Vous devez de nouveau rentrer les données pour votre carte cible BeagleBone :

- Host : adresse IP carte BeagleBone
- User : root
- Password : root.

Ne pas oublier de cocher "Save User" et "Save Password".

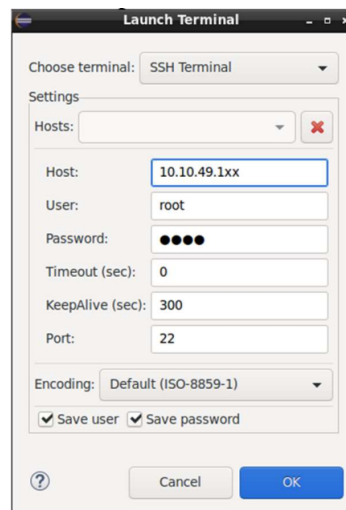
- OK

Remarque : Cette fenêtre Terminal intégrée à Eclipse est très pratique car cela évite d'utiliser un programme externe.

Une nouvelle fenêtre apparaît, vous devez saisir un "master password" (coffre-fort sécurisé), entrer root afin de se le rappeler facilement même si ce n'est pas sécurisé comme choix.

- OK

Puis vous devez autoriser la connexion. Enfin, une nouvelle fenêtre vous demande



si vous souhaitez entrer une question secrète pour pouvoir vous rappeler de ce "master password" sélectionner No.

1.6.6 Compiler et corriger le programme de test

Un programme de est est disponible sur le bureau.

Menu *File* > *New* > *Project* > *C/C++* > *C/C++ Project* > *C Managed Build*, puis donner le nom de projet "blink", laisser les valeurs par défaut, project type "Empty project" et Toolchains "Cross GCC", puis *Next*

Laisser les valeurs par défaut, *Debug* et *Release* cochés, puis *Next*.

Dans la case "cross compiler prefix" renseigner "arm-linux-gnueabi" (ne pas oublier le dernier -).

Dans la case "cross compiler path" renseigner "/opt/arm/bin/".

Puis *Finish*.

Pour intégrer le code source, vous pouvez tout simplement le copier en faisant un glisser-déplacer du dossier source qui se trouve sur le bureau dans le dossier 'blink' qui apparaît dans la vue "Project".

Si besoin de changer d'environnement :

Menu *Window* > *Perspective* > *Open Perspective* > *Other...* puis *C/C++*

Si besoin d'afficher l'explorateur :

Menu *Window* > *Show View* > *Project Explorer*

Tester la compilation de votre programme :

Menu *Project* > *Clean...* (éventuellement)

Menu *Project* > *Build All* le résultat apparaît dans la fenêtre "Console"

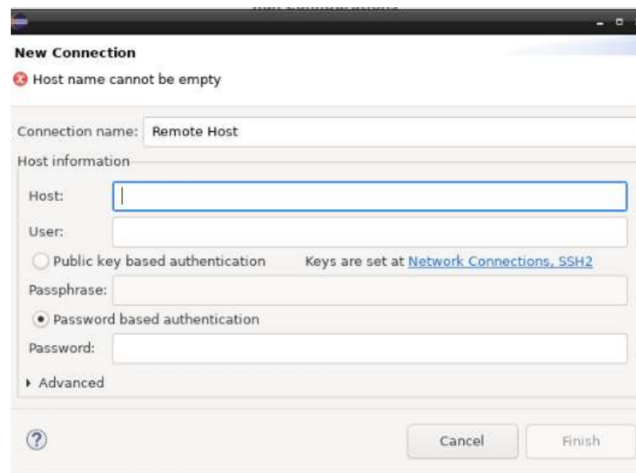
Le programme contient des erreurs que vous devrez corriger en fin de TP.

Vérifier l'accès SSH à la carte BeagleBone depuis Eclipse, si besoin reconfigurer l'accès.
Menu *Window > Show View > Other > Remote Systems* puis *Remote Systems*

Configurer le mode run (exécution du binaire généré) :

- Menu *Run > Run Configurations...*
- Ajouter une nouvelle configuration sous "C/C++ Remote Application" en double cliquant sur "C/C++ Remote Application"
- Les cases "Name", "Project" et "C/C++ Application" sont automatiquement préremplies
- Créer une nouvelle connexion distante en SSH avec la carte BeagleBone :

En face de Connexion sélectionner bouton New..., définir une connexion de type SSH



Host : adresse IP de votre carte BeagleBone

User : root

Password based authentication : root

- Pour les projets suivants, il suffira de sélectionner la connexion Remote Host dans le menu déroulant au lieu de Local.
- Dans "Remote Absolute File Path for C/C++ Applications" donner l'emplacement où sera téléchargé votre application sur la carte BeagleBone : `/root/blink`
- Dans "Commands to execute before application" indiquer que les droits du fichier qui sera téléchargé doivent être modifiés avant de l'exécuter : `chmod ugo+x /root/blink`

1.6.7 Tester, modifier et débbugger l'application blink

Lancer le mode run :

Menu *Run* > *Run Configurations...* (Ne pas oublier d'arrêter l'application en cliquant sur le carré rouge dans la fenêtre terminal.)

Il y a dans l'application blink des warnings et des erreurs que vous que vous devrez **identifier et corriger** :

- Warning sur "usleep" à la compilation (manque un include)
- Erreurs à l'exécution (chemin erroné pour les LED)
- Maladresses de codage : remplacer les appels systèmes sur les LED ("system (...)") par des `fopen(...)`, `fprintf(...)` et `fclose(...)`

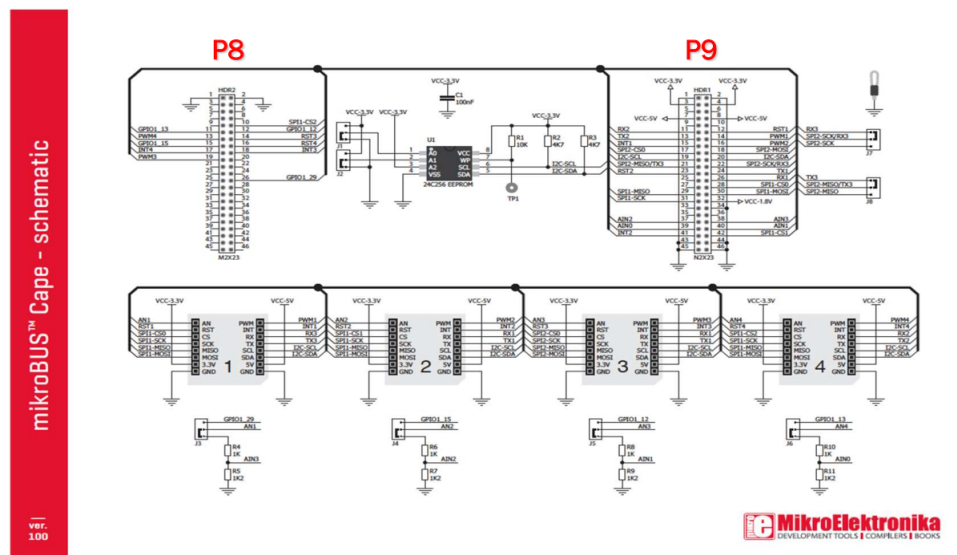
Afin de vous familiariser avec l'environnement, modifier le programme, tout en vérifiant son exécution sur la carte BeagleBone et les messages envoyés dans le terminal sur votre ordinateur.

Dans un second temps, nous ajouterons la configuration debug pour créer des points d'arrêt et visualiser le contenu et l'évolution des variables.

2 TP2 - GPIOs

2.1 Module Click ISEN Mikroe

Le module Click ISEN Mikroe se positionne sur l'emplacement 1 sur la carte d'extension Mikrobus qui elle-même est placée sur les connecteurs de la BeagleBone. Ce module permet de réaliser des exercices d'initiation mettant en œuvre le GPIO, une entrée analogique (ADC) et une sortie PWM.



Les périphériques disponibles sur cette carte sont :

- Une LED 3 couleurs intégrant dans le même composant une LED rouge, une LED verte et une LED bleue qui s'allument sur un état « 1 »,
- Un bouton poussoir qui donne un état « 0 » quand il est appuyé,
- Un potentiomètre permettant de générer une tension continue entre 0 et 1,8V pour l'acquisition par un ADC (1,8V correspond à la pleine échelle du convertisseur 12 bits)

IMPORTANT : Les scripts shell sont à réaliser en binôme car ils servent à configurer les entrées-sorties en fonction de configuration matérielle de la carte, par conséquent, il n'est pas nécessaire de les dupliquer ni de les lancer plusieurs fois.

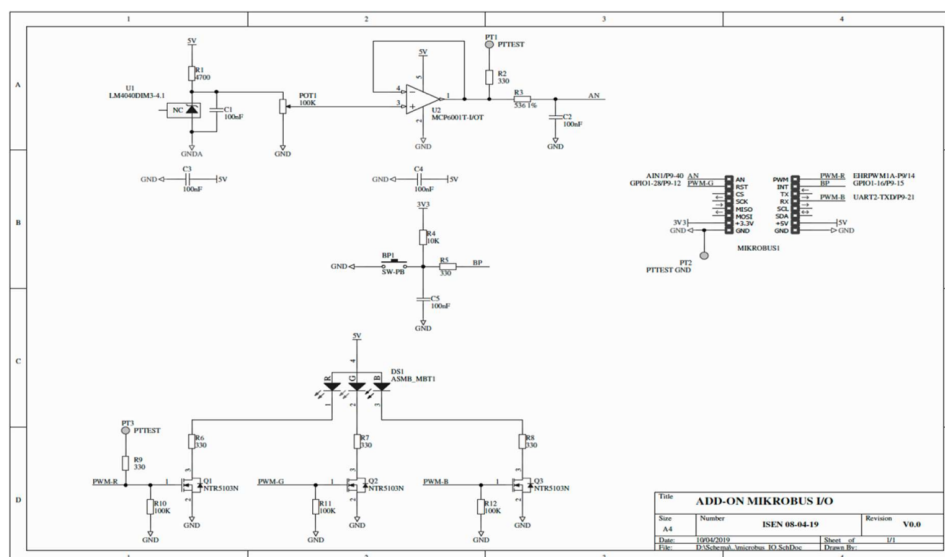
Le tableau ci-dessous précise les connections entre ces périphériques et les connecteurs de la BeagleBone.

Périphérique	Numéro des broches
LED rouge	P9-14 (GPIO1_18/EHRPWM1A)
LED verte	P9-12 (GPIO1_28)
LED bleue	P9-21 (GPIO0_3)
Bouton-poussoir	P9-15 (GPIO1_16)
Potentiomètre	P9-38 (AIN3)

2.2 Commande d'une LED par un bouton poussoir

Cet exercice consiste à allumer la LED rouge lorsque le bouton poussoir est appuyé.

En vous inspirant des exemples donnés en annexe 2 :



- Écrire avec l'éditeur de fichier texte en ligne de commande "nano" (ou "vim" ou directement depuis Eclipse ou depuis l'éditeur Atom) un script shell "init_IO.sh" qui initialise les GPIO, c.-à-d. les GPIO pour les 3 LEDS et pour le bouton poussoir.
- Rendre ce script exécutable et le lancer depuis le shell.
- Vérifier que le script a bien initialisé les entrées-sorties en allumant et éteignant la LED rouge et en lisant l'état du bouton poussoir avec des commandes shell en bash.

Après avoir créé un nouveau projet nommé LED_POUS, un dossier source et un fichier source nommé LED_POUS.c, vous modifierez le fichier source pour :

- Écrire une procédure "void allume_LED(void)" qui allume la LED rouge.
- Écrire une procédure "void eteint_LED(void)" qui éteint la LED rouge.
- Écrire une fonction "unsigned char etat_poussoir(void)" qui renvoie 0 quand le poussoir est appuyé et 1 sinon.
- Écrire la fonction "int main(void)" qui boucle à l'infini sur l'appel de "etat_poussoir" et l'appel de "allume_LED" ou "eteint_LED" suivant l'état du bouton poussoir.

Puis compiler et lancer votre programme sur la carte.

IMPORTANT : Ne pas oublier d'arrêter le programme avec le bouton en forme de carré rouge sur l'interface d'Eclipse.

2.3 Clignotement d'une LED

Cet exercice consiste à faire clignoter la LED verte.

Après avoir créé un nouveau projet nommé "clign_LEDV", un dossier source et un fichier source nommé "clign_LEDV.c", vous modifierez le fichier source pour :

- Écrire une fonction "void permut_LED(void)" qui permute l'état de la LED verte
- Écrire votre fonction "int main(void)" qui boucle à l'infini sur l'appel de "permut_LED" et une temporisation de 0,5 s

Puis compiler et lancer votre programme sur la carte.

3 TP3 - ADC et PWM

3.1 Lecture d'une entrée analogique

Contrôle des 3 LED en tout ou rien en fonction de la position/valeur du potentiomètre.

Cet exercice consiste à allumer une des 3 LED selon la valeur du potentiomètre :

- Si la valeur est entre 0% et <33% de la valeur max, vous devez allumer la LED rouge, les autres LED sont éteintes
- Si la valeur est entre 33% et <66% vous devez allumer la LED verte, les autres LED sont éteintes
- Si la valeur est entre 66% et 100% vous devez allumer la LED bleu, les autres LED sont éteintes

Pour cela, vous devez déterminer les valeurs numériques des débuts et fins des plages en fonction des niveaux de tension à l'entrée du convertisseur et de la valeur de la conversion (convertisseur avec 12 bits de résolution).

Vérifiez le résultat de la conversion avec la commande shell suivante :

```
cat /sys/bus/iio/devices/iio:device0/in_voltage3_raw
```

Après avoir créé un nouveau projet nommé `adc_LED`, un dossier source et un fichier source nommé `adc_LED.c`, vous modifierez le fichier source pour :

- Écrire une procédure `"void control_RVB (char R, char V, char B)"` qui contrôle en tout ou rien l'allumage des 3 LED, les paramètres R, V et B prenant la valeur de 1 ou de 0
- Écrire une fonction `"unsigned int read_ADC(void)"` qui convertit l'entrée analogique AIN3, retourne sa valeur numérique et l'affiche dans un terminal.
- Écrire la fonction `"int main(void) "` qui boucle à l'infini sur l'appel de `"read_ADC"` et `"control_RVB"`, les paramètres transmis à `"control_RVB"` dépendant du retour de `"read_ADC"`.

Puis compiler et lancer votre programme sur la carte.

3.2 Commande d'une LED en PWM

Cet exercice consiste à contrôler en PWM avec l'EHRPWM (contrôleur hardware) la LED rouge de façon proportionnelle à la position du potentiomètre c.à.d. de 0% à 100%.

Il nécessaire de bien comprendre le cours pour réaliser cette partie.

Dans un premier temps, il faut trouver la broche du connecteur P8 ou P9 qui est reliée à la LED rouge, puis trouver le sous-module correspondant : EHRPWM0, EHRPWM1 ou EHRPWM2 et la voie A ou B.

Ensuite, il faut trouver l'adresse du sous-module et avec cette adresse il faut trouver le point d'entrée au niveau du kernel pour accéder aux commandes PWM.

Modifier puis lancer votre script shell `"init_IO.sh"` pour effectuer les étapes suivantes en respectant cet ordre (voir le cours pour avoir le détail des commandes) :

- Changer le mode de la broche pilotant la LED rouge pour passer la broche en sortie PWM
- Désactiver la sortie EHRPWM correspondante
- Initialiser *period* à 1 ms, *duty_cycle* à 0 ms et *polarity* à normal
- Activer la sortie EHRPWM correspondante

Les valeurs de *period* et de *duty_cycle* sont des valeurs entières exprimées en nanosecondes.

Vérifier votre script en modifiant le *duty_cycle* avec une commande shell.

Après avoir créé un nouveau projet nommé LEDR_PWM, un dossier source et un fichier source nommé "LEDR_PWM.c", vous modifierez le fichier source pour :

- Réutiliser la fonction "unsigned int read_ADC(void)" qui convertit l'entrée analogique AIN3, retourne sa valeur numérique et l'affiche dans un terminal
- Créer une fonction "void mod_PWM (unsigned int duty)" qui règle le duty cycle du signal PWM en fonction du paramètre duty
- Écrire la fonction "int main(void)" qui boucle à l'infini sur l'appel de "read_ADC" et "mod_PWM". La fonction "mod_PWM" prend comme paramètre duty la valeur retournée par "read_ADC".

Puis compiler et lancer votre programme sur la carte.

4 TP4 - Duplication de processus

L'objectif de ce TP est d'appréhender les notions de programmation système en langage C sous l'environnement UNIX / LINUX. Nous aborderons les notions suivantes :

- Duplication de processus
- Communication par tubes

4.1 Fonction Fork

On vous demande de partir du squelette de programme suivant :

```

1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main () {
5
6     pid_t pidFork;
7     pidFork = fork();
8
9     printf("La valeur de retour de fork vaut %d\n", pidFork);
10
11     return(0);
12
13 }
```

Après avoir sauvegardé puis compilé le programme précédent on vous demande de lancer l'exécutable généré, qu'observez-vous à l'écran ?

En utilisant la fonction getpid() qui donne le PID du processus courant, complétez la fonction printf pour afficher en plus de pidFork la valeur du PID courant. Expliquez le phénomène.

Utilisez la valeur de retour de pidFork pour afficher dans le processus père "je suis dans le père" et dans le processus fils "je suis dans le fils".

Modifiez votre code pour que le processus fils affiche les nombres pairs de 0 à 20 et le processus père les nombres impairs de 1 à 19.

Faites une copie de votre code à ce stade car il servira de base au TP 5.

4.2 Communication entre processus

On souhaite à présent que le processus fils affiche à chaque itération 5 valeurs successives et le processus père 6 valeurs successives comme le montre le tableau ci-dessous :

Processus père	Processus fils
5, 6, 7, 8, 9, 10	0, 1, 2, 3, 4
16, 17, 18, 19, 20, 21	11, 12, 13, 14, 15

En reprenant le code de l'exercice précédent, utilisez la fonction pipe ainsi que les fonctions read et write pour communiquer d'un processus à l'autre la valeur courante du compteur.

5 TP5 - Signaux

L'objectif de ce TP est d'appréhender les notions de programmation système en langage C sous l'environnement UNIX / LINUX. Nous aborderons les notions suivantes :

- Duplication de processus
- Envoi et interprétation de signal
- Suspension de processus
- Communication par tubes

5.1 SIGKILL

Reprenez le code que vous aviez sauvé lors de la question 4.1.

Ajouter une temporisation de 1 seconde entre l'affichage de chaque nombre, on pourra utiliser la fonction `sleep(int s)` qui permet à un processus de se bloquer pendant `s` secondes.

Dans un nouveau terminal, utilisez la ligne de commande `ps -ef` pour afficher l'ensemble des processus actifs sur la machine. Qu'observez-vous ?

Afin de terminer l'exécution des processus créés par le programme, envoyez le signal 9 (SIGKILL) aux processus concernés. On rappelle que pour envoyer le signal 9 au processus de PID égal à 3150, il faut entrer dans le terminal la ligne de commande suivante :

```
kill -9 3150
```

Modifier votre programme afin que le processus père tue le processus fils quand le père affiche le nombre 9 en utilisant la fonction `kill`.

5.2 Utilisation de la fonction signal

Modifiez votre programme afin que le père soit bloqué à son début par une variable globale `ATTENTE_PERE` que vous initialiserez à 1 :

```
while(ATTENTE_PERE);
```

Modifiez le programme du fils afin qu'en fin d'exécution, celui-ci débloquent le processus père. Le déblocage sera effectué par l'envoi du signal `SIGUSR1` du processus fils au processus père. Le signal `SIGUSR1` sera interprété par le processus père comme le signal pour passer la variable globale `ATTENTE_PERE` à 0. Vous utilisez la fonction `signal(int signum, void (*handler))` qui permet de modifier le comportement d'un programme devant une interruption. Pour connaître le numéro du PID du père dans le fils on pourra utiliser la fonction `getppid()`...

5.3 Synchronisation de processus avec les signaux

En utilisant à présent le signal `SIGUSR2` et une variable globale `ATTENTE_FILS`, synchronisez les processus père et fils afin que les nombres soient affichés dans l'ordre croissant de 0 jusqu'à 20.

5.4 Attente passive

Afin d'éviter que les processus père et fils soient en attente active, on vous demande de reprendre l'exercice précédent en endormant les processus lorsqu'ils sont en attente. Vous devez donc supprimer les variables globales `ATTENTE_PERE` et `ATTENTE_FILS`, et utiliser la fonction `sigsuspend()` qui permet d'endormir un processus jusqu'à réception d'un signal particulier. Si l'on souhaite paramétrer l'attente d'un signal `SIGUSR1`, la fonction `sigsuspend()` est appelée de la façon suivante :

```
1 sigset_t mask1; // creation du masque d interruption
2 sigfillset (&mask1); // initialisation (toutes interruptions masquées)
3 sigdelset (&mask1, SIGUSR1); // demasquage de l interruption SIGUSR1
4
5 sigsuspend(&mask1); // suspend jusqu a reception de SIGUSR1
```

6 TP6 - Threads : le problème de l'orchestre

6.1 Présentation du problème

On propose de simuler le concert d'un orchestre et l'interaction avec les spectateurs.

Les règles du concert sont les suivantes :

- Les spectateurs entrent dans la salle de concert environ toutes les secondes
- L'orchestre commence à jouer que si NMIN spectateurs sont présents
- Aucun spectateur ne quitte la salle tant que l'orchestre n'a pas terminé de jouer
- Quand l'orchestre a terminé de jouer, les spectateurs applaudissent et quittent la salle

6.2 Programme initial

On vous demande de programmer en langage C le système décrit plus haut **en ne considérant pas pour l'instant les règles de synchronisation**.

Voici quelques indications :

- L'orchestre et les N spectateurs seront représentés par N+1 threads
- Vous utiliserez la bibliothèque <pthread.h> pour créer et joindre des threads
- Vous utiliserez la fonction printf() pour afficher les états des threads
- Vous utiliserez la fonction sleep(int n) pour temporiser chacun des threads de n secondes.
- Il y a N=4 spectateurs
- L'orchestre joue 10 secondes
- Le spectateur reste 5 secondes dans la salle de concert
- Afin de discerner les différents acteurs, les threads spectateurs sont créés avec 1 seconde de décalage.

Vous devez obtenir un affichage du type suivant :

```

1 Orchestre commence a jouer
2 Spectateur 1 entre dans la salle de concert
3 Spectateur 2 entre dans la salle de concert
4 Spectateur 3 entre dans la salle de concert
5 Spectateur 4 entre dans la salle de concert
6 Spectateur 5 entre dans la salle de concert
7 Spectateur 1 sort de la salle de concert
8 Spectateur 6 entre dans la salle de concert
9 Spectateur 2 sort de la salle de concert
10 Spectateur 3 sort de la salle de concert
11 Spectateur 4 sort de la salle de concert
12 Spectateur 5 sort de la salle de concert
13 Orchestre a fini de jouer
14 Spectateur 6 sort de la salle de concert

```

6.3 Nombre de spectateurs dans la salle

On vous demande de compléter le programme précédent afin d'afficher le nombre exact de spectateurs dans la salle au moment où un spectateur rentre et sort de la salle. Voici quelques indications :

- Vous utiliserez un variable globale nommée CPT de type int afin de comptabiliser le nombre actuel de spectateurs
- Vous utiliserez un sémaphore nommé VERROU afin de verrouiller l'accès à la variable globale CPT
- Vous utiliserez la bibliothèque <semaphore.h> pour implémenter les sémaphores
- Les sections critiques doivent être les plus courtes possibles

Vous devez obtenir un affichage du type suivant :

```

1 Orchestre commence a jouer
2 Spectateur 1 entre dans la salle de concert (1 spectateur(s) dans la salle)
3 Spectateur 2 entre dans la salle de concert (2 spectateur(s) dans la salle)
4 Spectateur 3 entre dans la salle de concert (3 spectateur(s) dans la salle)
5 Spectateur 4 entre dans la salle de concert (4 spectateur(s) dans la salle)
6 Spectateur 5 entre dans la salle de concert (5 spectateur(s) dans la salle)
7 Spectateur 1 sort de la salle de concert (4 spectateur(s) dans la salle)

```

```

8 Spectateur 6 entre dans la salle de concert (5 spectateur(s) dans la salle)
9 Spectateur 2 sort de la salle de concert (4 spectateur(s) dans la salle)
10 Spectateur 3 sort de la salle de concert (3 spectateur(s) dans la salle)
11 Spectateur 4 sort de la salle de concert (2 spectateur(s) dans la salle)
12 Spectateur 5 sort de la salle de concert (1 spectateur(s) dans la salle)
13 Orchestre a fini de jouer
14 Spectateur 6 sort de la salle de concert (0 spectateur(s) dans la salle)

```

6.4 Début concert

On souhaite à présent appliquer la règle de début de concert énoncée en début de sujet, à savoir l'orchestre commence quand NMIN spectateurs sont présents. On vous demande de compléter le programme précédent en mettant en œuvre cette règle. Voici quelques indications :

- Vous utiliserez un sémaphore nommé `DEB_CONCERT` permettant de bloquer le thread orchestre tant que le nombre de spectateurs requis n'est pas atteint
- Vous prendrez `NMIN = 3`
- Les sections critiques doivent être les plus courtes possibles

Vous devez obtenir un affichage du type suivant :

```

1 Spectateur 1 entre dans la salle de concert (1 spectateur(s) dans la salle)
2 Spectateur 2 entre dans la salle de concert (2 spectateur(s) dans la salle)
3 Spectateur 3 entre dans la salle de concert (3 spectateur(s) dans la salle)
4 Orchestre commence a jouer
5 Spectateur 4 entre dans la salle de concert (4 spectateur(s) dans la salle)
6 Spectateur 5 entre dans la salle de concert (5 spectateur(s) dans la salle)
7 Spectateur 1 sort de la salle de concert (4 spectateur(s) dans la salle)
8 Spectateur 6 entre dans la salle de concert (5 spectateur(s) dans la salle)
9 Spectateur 2 sort de la salle de concert (4 spectateur(s) dans la salle)
10 Spectateur 3 sort de la salle de concert (3 spectateur(s) dans la salle)
11 Spectateur 4 sort de la salle de concert (2 spectateur(s) dans la salle)
12 Spectateur 5 sort de la salle de concert (1 spectateur(s) dans la salle)
13 Spectateur 6 sort de la salle de concert (0 spectateur(s) dans la salle)
14 Orchestre a fini de jouer

```

6.5 Fin de concert

n souhaite enfin appliquer la règle de fin de concert énoncée en début de sujet, à savoir tous les spectateurs attendent que l'orchestre finisse le concert pour sortir de la salle. On vous demande de compléter le programme précédent en mettant en œuvre cette règle. Voici quelques indications :

- Vous utiliserez un sémaphore nommé `FIN_CONCERT` permettant de bloquer les threads spectateurs tant que le concert n'est pas fini
- Vous supprimerez la temporisation de 5 secondes des spectateurs pendant le concert qui, du coup, n'a plus lieu d'être !
- Vous pourrez vous inspirer de l'exemple sur le principe du « Rendez-vous » vu en cours
- Les sections critiques doivent être les plus courtes possibles

Vous devez obtenir un affichage du type suivant :

```

1 Spectateur 1 entre dans la salle de concert (1 spectateur(s) dans la salle)
2 Spectateur 2 entre dans la salle de concert (2 spectateur(s) dans la salle)
3 Spectateur 3 entre dans la salle de concert (3 spectateur(s) dans la salle)
4 Orchestre commence a jouer
5 Spectateur 4 entre dans la salle de concert (4 spectateur(s) dans la salle)
6 Spectateur 5 entre dans la salle de concert (5 spectateur(s) dans la salle)
7 Spectateur 6 entre dans la salle de concert (6 spectateur(s) dans la salle)
8 Orchestre a fini de jouer
9 Spectateur 1 sort de la salle de concert (5 spectateur(s) dans la salle)
10 Spectateur 2 sort de la salle de concert (4 spectateur(s) dans la salle)
11 Spectateur 3 sort de la salle de concert (3 spectateur(s) dans la salle)
12 Spectateur 4 sort de la salle de concert (2 spectateur(s) dans la salle)
13 Spectateur 5 sort de la salle de concert (1 spectateur(s) dans la salle)
14 Spectateur 6 sort de la salle de concert (0 spectateur(s) dans la salle)

```

6.6 Avec les processus

On vous demande de réaliser un programme similaire à la partie précédente mais en utilisant cette fois la duplication de processus. Les règles de codage sont les suivantes :

- Vous utiliserez la fonction `fork()` pour réaliser la duplication de processus
- Le Processus père représente l'orchestre
- Le Processus fils représente les spectateurs (il y aura un sel processus pour tous les spectateurs)
- Les règles de synchronisation sont les mêmes que pour la partie précédente
- Pour synchroniser les processus vous pourrez utiliser au choix les tubes ou les signaux associés par exemple avec la fonction `sigsuspend()`